

Institute of Business Administration
School of Mathematics and Computer Science

CSE145 – Introduction to Computing

Credit hour structure	(3,0,0) = (Theory, Lab, Total)
Pre-requisite(s)	None
Co-requisite(s)	Introduction to Programming
Course Lead / Email	Waseem Arain / warain@iba.edu.pk
Version	August

Course Description

This course introduces computer science as a language for solving problems, not merely a language for programming. Students explore the matrix-multiplication problem space to build abstraction and divide-and-conquer thinking, then descend into the hardware layer – binary logic, gates, adder circuits, memory and bandwidth – culminating in a Von Neumann-style CPU. Number bases and modular arithmetic are developed both as mathematical tools and as the representations needed to build real circuits. Before the capstone, the course turns to the theory of computing – regular languages, Turing machines, universality, and intractability – to establish what any computer, however built or simulated, can and cannot do. The course closes with its signature project: a working TOY machine built bottom-up in Logisim, which students can then measure against the theoretical limits just covered.

Course Learning Outcomes (CLOs)

Upon successful completion of this course, students will be able to:

CLO	Title	Description
CLO-1	Computational Thinking & Abstraction	Apply abstraction and notional-machine thinking to a problem space (matrix multiplication), comparing naive and divide-and-conquer solutions and reasoning about sequential vs distributed approaches to a solution.
CLO-2	Number Systems & Digital Logic	Perform arithmetic and translation across binary, octal, hexadecimal and modular arithmetic, and apply these representations to design basic combinational logic (gates, adders) in Logisim.
CLO-3	Machines, Architecture & the Limits of Computation	Describe the Von Neumann architecture and CPU components and build a working machine bottom-up in Logisim; describe regular languages, Turing machines, universality/computability, and intractability to explain what any computer – built or simulated – can and cannot do.
CLO-4	Professional Tools & Computing Practice	Use modern professional tools (version control, shell, editors) to support development and debugging, and engage with emerging technologies and the professional/ethical context of computing.

CLO–PLO Mapping

CLO	Mapped PLOs (Seoul Accord)
CLO-1	PLO-2, PLO-3, PLO-10
CLO-2	PLO-2, PLO-4
CLO-3	PLO-2, PLO-3, PLO-4, PLO-5, PLO-6
CLO-4	PLO-5, PLO-7, PLO-8, PLO-9

Course Structure & Tentative Topics

The course is organized into five parts across a 14-week semester. The capstone project begins in Part III and continues through Part V (see Capstone Project below for the full schedule).

Part I — Context & Computational Thinking (Weeks 1–2)

Week(s)	Topic
1	Computational thinking & abstraction; CS as a language for solving problems
1	Logisim orientation (tool tutorial)
2	Problem space: matrix multiplication; edge cases & testing

Part II — Solvable Problems & Number Systems (Weeks 3–5)

Week(s)	Topic
3	Number bases I – binary, translations, binary arithmetic
4	Number bases II – octal/hex, translations, steganography; modular arithmetic & encryption
5	Sequential vs distributed thinking; compute capacity/memory/bandwidth; sorting, searching & basic data structures

Part III — The Hardware Layer (Weeks 6–8)

Week(s)	Topic
6	Binary logic, gates, Boolean algebra
7	Adder circuits (2-bit → 16-bit); memory-to-processor bandwidth
8	Von Neumann architecture, CPU components, registers, program counter; instruction sets (simple/reduced/complex), control lines

Part IV — Theory of Computing: The Limits of Machines (Weeks 9–11)

Week(s)	Topic
9	Regular expressions & DFAs – what finite machines can and cannot recognize

Week(s)	Topic
10	Turing machines, universality & computability
11	Intractability – P vs NP, NP-completeness

Part V — Capstone Integration & Demos (Weeks 12–14)

Week(s)	Topic
12–13	Capstone: integrate, test & debug the assembled TOY machine
14	Closing reflection: your TOY vs. the Turing machine
14	Optional stretch: a matrix-multiplication unit in Logisim (bonus)
14	Final demos & presentations

Capstone Project

This project begins in Week 6 and unfolds in milestones across the semester, tracking the Part III lectures one-to-one. By Part V, the focus shifts to integration, testing, and debugging – not first-time construction.

Week(s)	Milestone	Notes
6	M1: Gates & combinational logic: basic logic gates plus a half/full adder in Logisim	Aligned with this week's lecture on binary logic and gates
7	M2: A scaled adder (2-bit → 16-bit) forming the ALU's core	Aligned with this week's lecture on adder circuits
8	M3: Registers, memory cells, and a control-unit skeleton	Aligned with this week's lecture on architecture and instruction sets
9–11	M4: Assemble the fetch-decode-execute cycle into a working TOY	Continues at a lighter pace alongside the theory-of-computing lectures
12–13	M5: Integration, full-program testing, and debugging	Focus shifts from building to verifying and refining your machine
14	M6: Final demos; optional matrix-multiplication bonus for teams that finish M5 early	Bonus component, not required for the core grade

The closing lecture connects your finished build back to Part IV: by the Church-Turing thesis, the TOY is in principle equivalent in computational power to any computer that has ever existed – the same idea that underpins virtualization – yet it remains bound by the same undecidability and intractability results covered earlier in the course.

Computing Ecosystem Literacy

Mandatory tutorials, examined through quizzes and assignments:

- Logisim orientation (Week 1)
- Street-smart computing
- Version control (Git)

- Editors
- Data wrangling
- Shell tools & scripting
- Command-line environment
- Security & cryptography

Emerging Technologies

Workshop/guest-lecture topics:

- Artificial Intelligence and Machine Learning
- Internet of Things
- Cloud Computing
- Cyber Security

Reference Books & Resources

- R. Sedgewick and K. Wayne, Computer Science: An Interdisciplinary Approach, 1st ed., Addison-Wesley Professional, 2016 – primary text, chapters 5–7.
- University of London (Coursera), “How Computers Work” – supplementary.
- D. Brylow and G. J. Brookshear, Computer Science: An Overview, 13th ed., Pearson, 2021 – supplementary.
- Logisim (<http://www.cburch.com/logisim/>) used throughout Parts III and V.

Assessment Plan

Tentative – problem sets and assignments remain a must-pass component regardless of credit weight.

Component	Percentage
Quizzes	15%
Problem sets (matrix multiplication, number bases, automata/Turing/NP)	20%
Mid-Term Exam	20%
Capstone project (milestone-graded, Weeks 6-14: TOY build M1-M5; matmul bonus optional)	20%
Final Exam	25%

Quiz Policy

Quizzes will be conducted at unannounced times throughout the semester and therefore require regular attendance and active participation in class. As there will be a substantial number of quizzes over the term, missing a few will not significantly affect a student's overall standing; however, habitually missing quizzes will negatively impact the final grade.

Late Submissions & Makeup Policy

Late submissions lose 20% credit per day after the due date; after that, submission is mandatory without credit. Documented medical grounds shift the credit to the final weightage. No makeup exams.